

Ordem Dev

Do Código à Produção



O que é o ORDEM DEV



Do **código**
ao **sistema real**



Ferramentas
do **mercado**



Engenharia
de **software**



Por onde vamos navegar



- ~~Versionamento com Git~~
- ~~Construção de APIs~~
- Persistência de **dados**
- Integração com **frontend**
- **Deploy** de aplicações
- Testes de **carga**



Gabriel Souza Paes

Ciências da Computação

3 publicações científicas

1 projeto virando produto

Desenvolvedor frontend

WordPress, PHP, JS, SCSS
& HTML

Ordem Dev - Do Código à Produção

Eu, o apresentador

Vivência de mercado
dev PJ • remoto

Projetos reais
e **colaboração** em **equipe**

Código que **precisa**
funcionar em **produção**

Minha missão hoje:

mostrar o **caminho**
do **código** à **produção**



GitHub

**Vamos continuar
a conversa?**

LinkedIn



Se você não estava aqui no **Módulo 1**

- **Git** é a máquina do tempo do código. Você viaja entre versões sem medo de quebrar nada.
- **GitHub** é o repositório remoto — onde o código vive em equipe, não só na sua máquina.
- **Commit** = foto do código naquele instante.
Clone = copiar o projeto completo com toda a história.



Se você não estava aqui no **Módulo 2**

- **API** é o canal entre sistemas. O servidor responde porque você mandou.
- **Race condition:** 10 pessoas, 1 assento, todas confirmaram. Código certo $\neq$ sistema seguro
- A API rodava **só na rede local**. Hoje ela vai para a internet.



Você fechou o terminal.

Cadê as reservas?

Tudo que guardamos em variáveis vive na RAM.

Processo encerrado — memória liberada. Dados: gone.

RAM (memória)	Disco (arquivo)
Rápido	Persistente
Morre com o processo	Sobrevive ao restart
Onde a API vive agora	Onde vamos salvar

Sem banco de dados. Por enquanto.

Banco de dados adiciona: instalação, configuração, conexão, autenticação, driver. Isso mata iniciantes.

- Um arquivo **reservations.json** resolve o problema agora.
- É lento em escala? Sim. **Mas escala não é o problema hoje.**
- O conceito é o mesmo — **ler, escrever, persistir**. O banco vem no módulo certo.



```
import json, os

DB = 'reservations.json'

# Lê os dados do disco

def load():

    if not os.path.exists(DB):

        return []

    with open(DB) as f:

        return json.load(f)

# Salva os dados no disco

def save(data):

    with open(DB, 'w') as f:

        json.dump(data, f, indent=2)
```

Ler. Alterar. Salvar.

Feche o servidor. Abra de novo. Os dados continuam lá.

A API saindo da sua máquina.

Enquanto a API roda só no seu computador, ela não existe para o mundo. Deploy é colocar o código num servidor que nunca desliga.

Local

`http://localhost:8000`

Só você acessa

Morre quando você fecha

Produção

`https://cine-fila.onrender.com`

Qualquer pessoa acessa

Fica no ar 24/7



3 cliques. URL no ar.

Plataformas gratuitas que lêem direto do GitHub e sobem o servidor automaticamente.

- 01 Faça login com sua conta GitHub em render.com ou railway.app
- 02 Conecte o repositório `api-cine`
- 03 Defina o comando de start: `uvicorn base.main:app --host 0.0.0.0 --port $PORT`
- 04 Deploy. Em 2 minutos você tem uma URL pública.

Agora qualquer um pode atacar.

- 1. Abra o browser no celular — sem instalar nada.
- 2. Acesse: <https://api-cine-0yls.onrender.com/docs>
- 3. Encontre **POST /reserve**. Preencha seu nome.
- 4. Clique Execute ao mesmo tempo que todo mundo.
- 5. Abra **/reservations** e conte quantas reservas aparecem



I know Deploy: a API está no ar.

- Os dados sobrevivem ao terminal.
- URL funciona de qualquer dispositivo.
- Você entendeu a diferença entre rodar e publicar.



Por hoje é só



Dúvidas? Feedbacks?